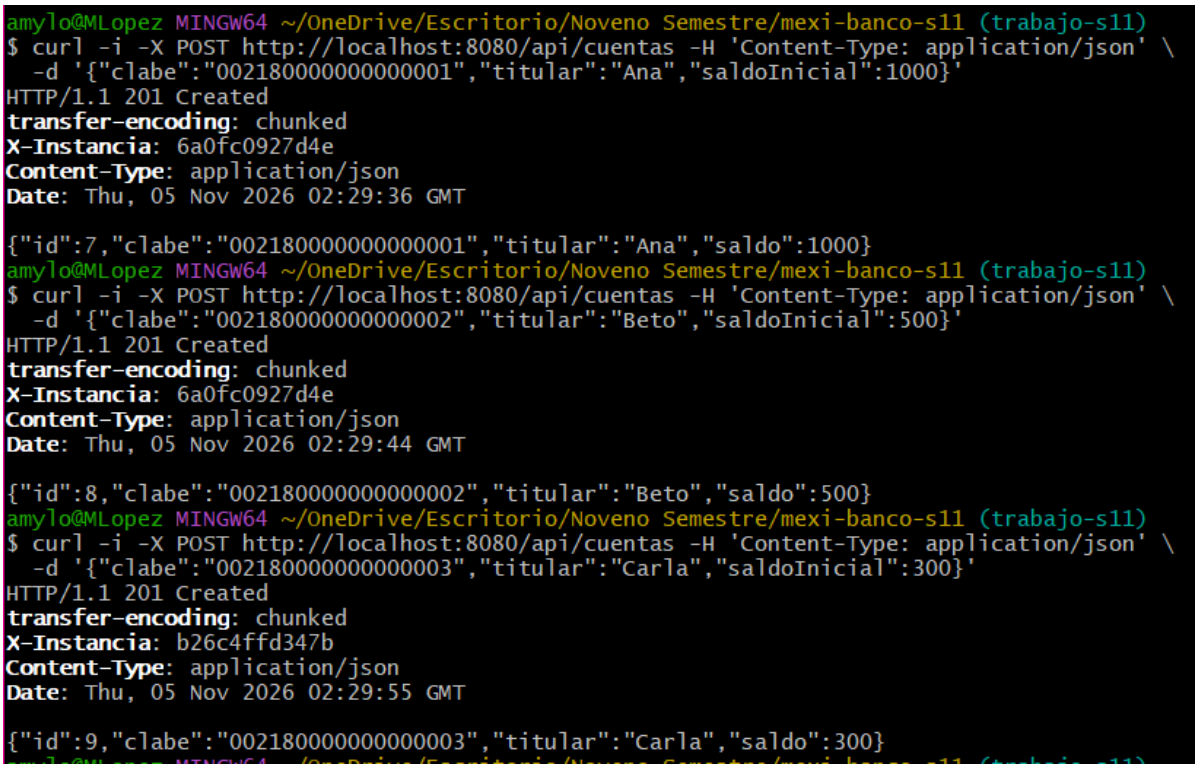
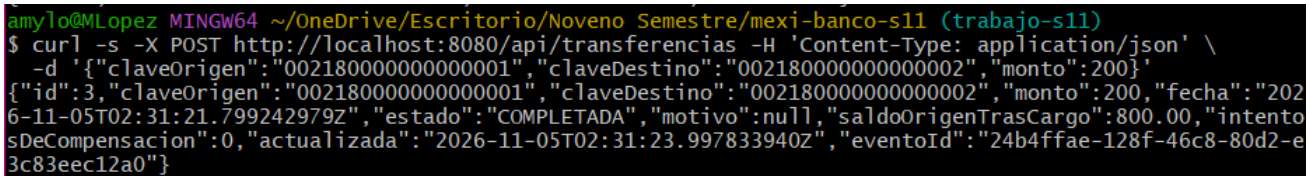
1. Punto de partida

Primero levanté los servicios de Mexi Banco con Docker. Redis respondió PONG y la colección de avisos en MongoDB estaba vacía. Después creé las cuentas de Ana ($1,000), Beto ($500) y Carla ($300). La primera transferencia fue de $200 de Ana a Beto y terminó con estado COMPLETADA. El saldo de Ana pasó a $800.



Creación de las tres cuentas mediante el endpoint de cuentas



Transferencia inicial de $200 con estado COMPLETADA

Antes de configurar la caché consulté diez veces el saldo de Ana. El campo leidaEn cambió en cada respuesta porque todavía se leía PostgreSQL. El registro de logs mostró 12 lecturas acumuladas (las diez de la prueba y otras anteriores); Redis registró cero hits y cero misses.

Indicador antes de Redis	Resultado
Consultas consecutivas al saldo	10
Lecturas registradas en logs	12 (acumuladas)
keyspace_hits	0
keyspace_misses	0

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
done
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:13.282819440Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:13.630278419Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:13.927223816Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:14.225386059Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:14.508295246Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:14.823800744Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:15.146266079Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:15.426805687Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:15.685856974Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:33:16.122620126Z"
}

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep -c "leida de Postgres"
12

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

Diez respuestas, conteo de lecturas a PostgreSQL y estadísticas iniciales de Redis

2. Caché de saldos con Redis

Cambios en el servicio cuenta

En ConfiguracionCache.java activé el almacenamiento en caché con @EnableCaching, un TTL de 30 segundos, serialización JSON y transactionAware()

En ConsultaDeSaldo.java usé @Cacheable para el GET que muestra el saldo

En CuentaService.java agregué @CacheEvict a cargar y abonar para borrar la copia después de cambiar el saldo

La consulta que autoriza cargos siguió leyendo PostgreSQL.

@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")

Consulta compartida, JSON y borrado

Hice seis consultas consecutivas al saldo de Ana y observé el mismo leidaEn aunque respondió más de una instancia de cuenta, lo que mostró que compartían Redis. También comprobé que Redis guardaba el saldo como JSON y que quedaban 25 segundos de TTL.

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001; echo

docker compose exec -T redis redis-cli GET cuentas::002180000000000001

docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:55:21.103741244Z"
}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T02:55:21.103741244Z"
}
25
```

El GET y el valor JSON en Redis, con TTL de 25 segundos

Luego transferí $10 de Ana a Beto. Redis registró DEL para ambas cuentas, la consulta EXISTS de Ana devolvió 0 y el siguiente GET mostró el saldo actualizado de $790.

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001; echo

curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-05T03:08:37.421274768Z"
}
{"id":4,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10,"fecha":"2026
-11-05T03:08:39.849740517Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":790.00,"intentos
DeCompensacion":0,"actualizada":"2026-11-05T03:08:41.611637783Z","eventoId":"f6246c01-65fb-4b50-be2a-a2
9e557addd1"}
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
0

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":790.00,"leidaEn":"2026-11-05T03:09:09.580495879Z"
}
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$
```

```
MINGW64:/c/Users/amylo/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11
1793848081.428116 [0 127.0.0.1:40756] "ping"
1793848086.575842 [0 127.0.0.1:59002] "ping"
1793848091.858993 [0 127.0.0.1:52106] "ping"
1793848096.973390 [0 127.0.0.1:52116] "ping"
1793848102.097683 [0 127.0.0.1:34660] "ping"
1793848107.219610 [0 127.0.0.1:34670] "ping"
1793848112.341359 [0 127.0.0.1:38508] "ping"
1793848117.404363 [0 172.24.0.6:49032] "GET" "cuentas::002180000000000001"
1793848117.423765 [0 172.24.0.6:49032] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titu
lar\":\"Ana\",\"saldo\":800.00,\"leidaEn\":\"2026-11-05T03:08:37.421274768Z\"}" "PX" "30000"
1793848117.466275 [0 127.0.0.1:38524] "ping"
1793848118.972272 [0 172.24.0.11:54548] "GET" "cuentas::002180000000000001"
1793848119.028848 [0 172.24.0.10:51282] "GET" "cuentas::002180000000000002"
1793848119.836650 [0 172.24.0.10:51282] "SET" "cuentas::002180000000000002" "{\"clabe\":\"002180000000000002\",\"tit
ular\":\"Beto\",\"saldo\":700.00,\"leidaEn\":\"2026-11-05T03:08:39.781966734Z\"}" "PX" "30000"
1793848120.771540 [0 172.24.0.6:49032] "DEL" "cuentas::002180000000000001"
1793848121.375174 [0 172.24.0.11:54548] "DEL" "cuentas::002180000000000002"
1793848122.592892 [0 127.0.0.1:40488] "ping"
1793848127.703587 [0 127.0.0.1:40494] "ping"
1793848132.830203 [0 127.0.0.1:55632] "ping"
1793848137.919021 [0 127.0.0.1:55640] "EXISTS" "cuentas::002180000000000001"
1793848137.933294 [0 127.0.0.1:55648] "ping"
1793848143.041524 [0 127.0.0.1:47310] "ping"
1793848148.155598 [0 127.0.0.1:47316] "ping"
1793848149.569216 [0 172.24.0.11:54548] "GET" "cuentas::002180000000000001"
1793848149.582567 [0 172.24.0.11:54548] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"tit
ular\":\"Ana\",\"saldo\":790.00,\"leidaEn\":\"2026-11-05T03:09:09.580495879Z\"}" "PX" "30000"
1793848153.297494 [0 127.0.0.1:37896] "ping"
```

Transferencia de $10, eliminación de caché, nuevo saldo y comandos observados en MONITOR

Porcentaje de aciertos (hit ratio)

Escenario	Hits	Misses	Hit ratio
A. Cien consultas seguidas	99	1	99.00 %
B. Transferencia cada 10 consultas	100	20	83.33 %

En el escenario A casi todas las consultas encontraron la copia del saldo, en el B, cada transferencia borró las entradas de Redis y eso generó más fallos de caché. Calculé el hit ratio como hits / (hits + misses)

```
}
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$
docker compose exec -T redis redis-cli DEL cuentas::002180000000000001
docker compose exec -T redis redis-cli CONFIG RESETSTAT

for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done

docker compose exec -T redis redis-cli INFO stats | grep keyspace
0
OK
keyspace_hits:99
keyspace_misses:1

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
```

Escenario A

```
OK
keyspace_hits:99
keyspace_misses:1

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$
docker compose exec -T redis redis-cli DEL cuentas::002180000000000001
docker compose exec -T redis redis-cli CONFIG RESETSTAT

for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001

  if (( i % 10 == 0 )); then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias \
      -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
  fi
done

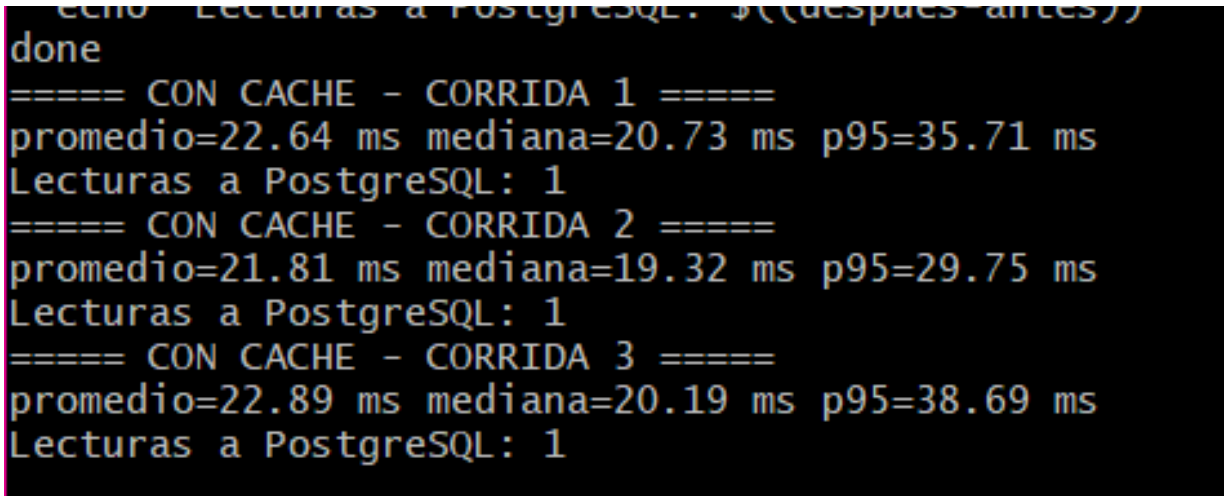
docker compose exec -T redis redis-cli INFO stats | grep keyspace
0
OK
keyspace_hits:100
keyspace_misses:20
```

Escenario B

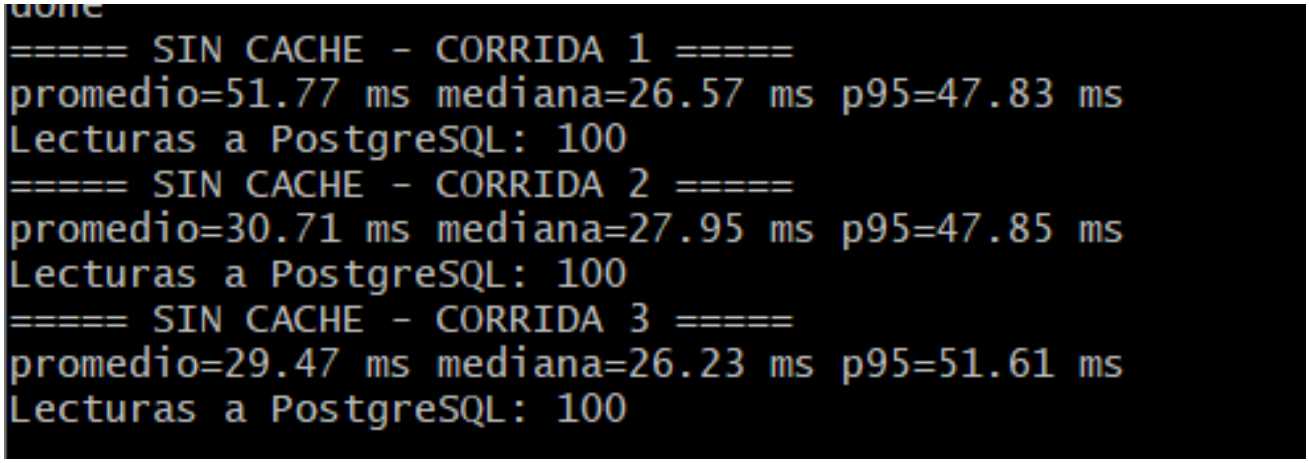
Comparación de latencia con y sin caché

Repetí tres corridas de 100 consultas para cada configuración y registré promedio, mediana, percentil 95 y lecturas a PostgreSQL. Los tiempos variaron entre corridas, pero con caché las medianas fueron menores en las tres.

Caché	Corrida	Promedio	Mediana	p95	Lecturas PG
Sí	1	22.64	20.73	35.71	1
Sí	2	21.81	19.32	29.75	1
Sí	3	22.89	20.19	38.69	1
No	1	51.77	26.57	47.83	100
No	2	30.71	27.95	47.85	100
No	3	29.47	26.23	51.61	100



Tres mediciones de latencia con caché habilitada



Tres mediciones de latencia con caché deshabilitada

Además de reducir los tiempos de consulta en mis pruebas, el principal beneficio fue disminuir el trabajo de lectura de PostgreSQL, para que el motor pueda concentrarse en las operaciones que modifican dinero

3. Notificaciones en MongoDB

Completé los métodos de BitacoraDeAvisos.java para armar documentos según el canal, registrar avisos sin duplicarlos, crear el índice clabe_fecha y consultar los diez avisos más recientes. En el

consumidor de RabbitMQ cambié la escritura anterior por bitacora.registrar(evento). Los montos y saldos se almacenaron en Decimal128, sin utilizar double

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-05T13:41:23.738Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos            : [avisos] indice clabe_fecha listo en avisos

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
```

Confirmación en logs de que se creó el índice clabe_fecha

Tres tipos de aviso

Tras hacer una transferencia de Ana a Beto y otra de Ana a Carla, MongoDB mostró cuatro documentos,dos SMS para Ana, un CORREO para Beto y un PUSH para Carla. Cada documento conservó los campos comunes y añadió solo los correspondientes a su canal

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: '78902ab9-1c88-4208-9a28-ee7040ac8ed7:002180000000000001',
    eventoId: '78902ab9-1c88-4208-9a28-ee7040ac8ed7',
    transferenciaId: Long('15'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('10'),
    saldo: Decimal128('770.00'),
    fecha: ISODate('2026-11-05T13:43:32.299Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 10, saldo resultante 770.00'
  },
  {
    _id: '78902ab9-1c88-4208-9a28-ee7040ac8ed7:002180000000000002',
    eventoId: '78902ab9-1c88-4208-9a28-ee7040ac8ed7',
    transferenciaId: Long('15'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('10'),
    saldo: Decimal128('730.00'),
    fecha: ISODate('2026-11-05T13:43:32.299Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 10, saldo resultante 730.00'
  },
  {
    _id: 'a3048388-45d4-44e8-ae66-7cb84d5ee24d:002180000000000001',
    eventoId: 'a3048388-45d4-44e8-ae66-7cb84d5ee24d',
    transferenciaId: Long('16'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('10'),
    saldo: Decimal128('760.00'),
    fecha: ISODate('2026-11-05T13:44:24.939Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 10, saldo resultante 760.00'
  },
  {
    _id: 'a3048388-45d4-44e8-ae66-7cb84d5ee24d:002180000000000003',
    eventoId: 'a3048388-45d4-44e8-ae66-7cb84d5ee24d',
    transferenciaId: Long('16'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('10'),
    saldo: Decimal128('310.00'),
    fecha: ISODate('2026-11-05T13:44:24.939Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 10, saldo resultante 310.00'
  }
```

Cuatro documentos en MongoDB con los canales SMS, CORREO y PUSH

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}},
 {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '78902ab9-1c88-4208-9a28-ee7040ac8ed7:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'a3048388-45d4-44e8-ae66-7cb84d5ee24d:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Consulta que filtra exclusivamente los dos avisos SMS de Ana

Últimos diez avisos e índices

Después de doce transferencias de $1, el endpoint GET /api/notificaciones/{clabe} devolvió diez avisos CORREO de Beto, del más nuevo al más antiguo. Verifiqué los índices _id_ y clabe_fecha: { clabe: 1, fecha: -1 }

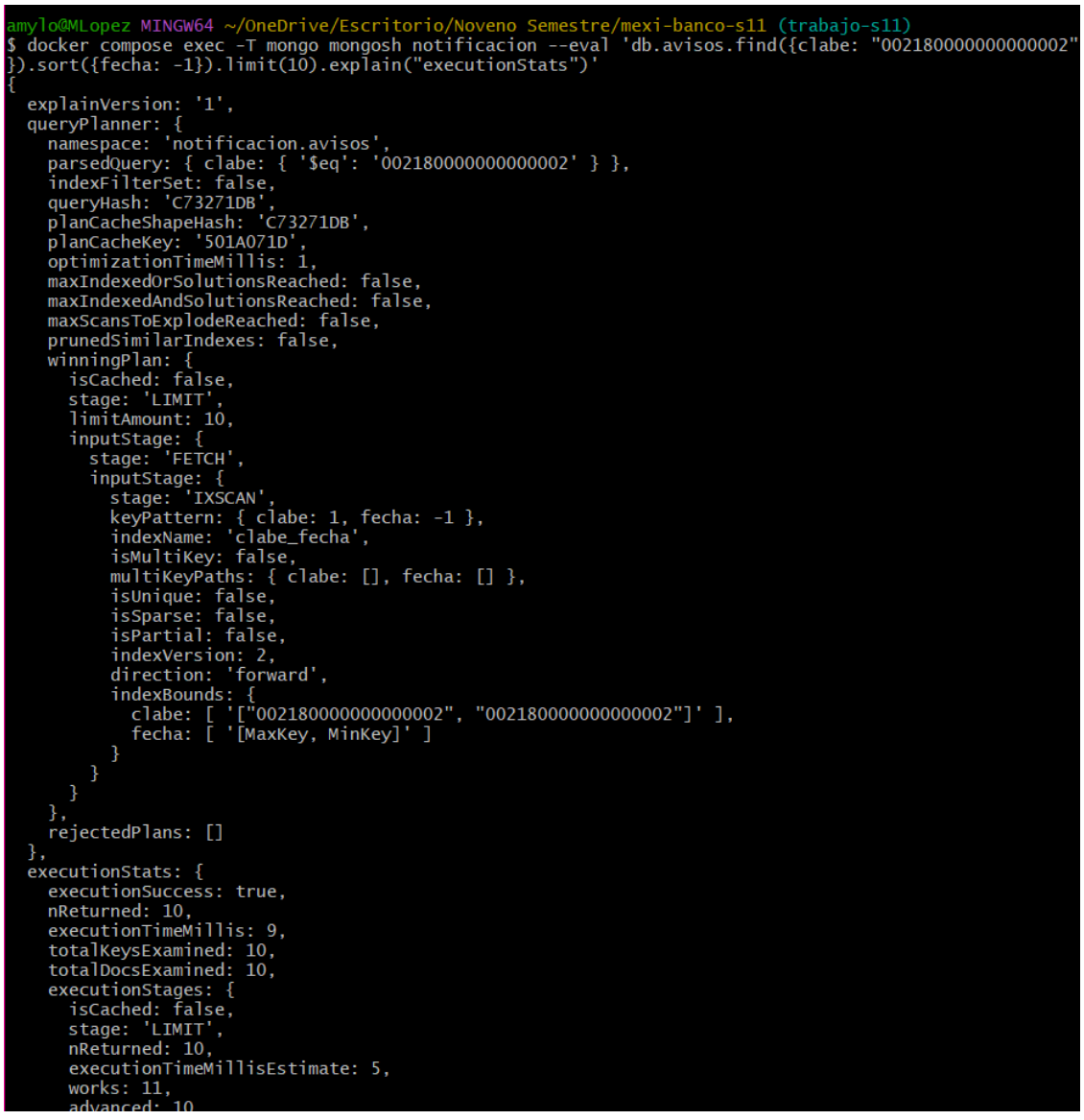
```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002
[{"_id":"ecf00371-e98a-42c5-bcef-94c64f937dbc:002180000000000002","eventoId":"ecf00371-e98a-42c5-bcef-9
4c64f937dbc","transferenciaId":28,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":
1,"saldo":742.00,"fecha":"2026-11-05T13:48:57.462+00:00","canal":"CORREO","correo":"beto@correo.mx","as
unto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 742.00"},{
"_id":"3ebdd55b-cd36-4923-8624-4292b5bb64fe:002180000000000002","eventoId":"3ebdd55b-cd36-4923-8624-429
2b5bb64fe","transferenciaId":27,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,
"saldo":741.00,"fecha":"2026-11-05T13:48:56.692+00:00","canal":"CORREO","correo":"beto@correo.mx","asun
to":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 741.00"},{"_
id":"563a2f9c-7f94-48fc-8355-31e36487caf3:002180000000000002","eventoId":"563a2f9c-7f94-48fc-8355-31e36
487caf3","transferenciaId":26,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"s
aldo":740.00,"fecha":"2026-11-05T13:48:55.709+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto
":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 740.00"},{"_id
":"19e457c6-46d9-418b-b89d-460badabbb71:002180000000000002","eventoId":"19e457c6-46d9-418b-b89d-460bada
bbb71","transferenciaId":25,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"sal
do":739.00,"fecha":"2026-11-05T13:48:54.836+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":
"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 739.00"},{"_id":
"0edcec5c-97e4-434c-9987-48c19a3a6a76:002180000000000002","eventoId":"0edcec5c-97e4-434c-9987-48c19a3a6
a76","transferenciaId":24,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo
":738.00,"fecha":"2026-11-05T13:48:53.810+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"R
ecibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 738.00"},{"_id":"7
0c2423a-59b1-46ba-9d0f-0b4827371ff4:002180000000000002","eventoId":"70c2423a-59b1-46ba-9d0f-0b4827371ff
4","transferenciaId":23,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":
737.00,"fecha":"2026-11-05T13:48:52.904+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Rec
ibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 737.00"},{"_id":"3ff
e03ce-c230-4cd4-bb63-e3846c04ae90:002180000000000002","eventoId":"3ffe03ce-c230-4cd4-bb63-e3846c04ae90"
,"transferenciaId":22,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":73
6.00,"fecha":"2026-11-05T13:48:52.010+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recib
iste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 736.00"},{"_id":"9387b
00e-80e4-4920-b5d9-1eafd622e3ad:002180000000000002","eventoId":"9387b00e-80e4-4920-b5d9-1eafd622e3ad","
transferenciaId":21,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":735.
00,"fecha":"2026-11-05T13:48:50.626+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibis
te una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 735.00"},{"_id":"98c796e
6-aa04-4903-a27a-4dfeb3c5ccf5:002180000000000002","eventoId":"98c796e6-aa04-4903-a27a-4dfeb3c5ccf5","tr
ansferenciaId":20,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":734.00
,"fecha":"2026-11-05T13:48:49.488+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste
 una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 734.00"},{"_id":"1226c204-
95a6-418a-a529-d8d9cd3b90f0:002180000000000002","eventoId":"1226c204-95a6-418a-a529-d8d9cd3b90f0","tran
sferenciaId":19,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":733.00,"
fecha":"2026-11-05T13:48:48.525+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste u
na transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 733.00"}]
```

El endpoint devuelve diez avisos recientes de Beto

Consulta con y sin índice

Consulta	Plan de ejecución	Documentos examinados
Con índice	IXSCAN	10
Sin índice	COLLSCAN	28

Con el índice, MongoDB encontró los avisos examinando diez documentos. Al forzar una exploración sin índice, examinó 28. Aunque la colección todavía era pequeña, esta prueba muestra por qué el índice ayuda cuando crece la cantidad de avisos.



```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"
}).sort({fecha: -1}).limit(10).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { clabe: 1, fecha: -1 },
          indexName: 'clabe_fecha',
          isMultiKey: false,
          multiKeyPaths: { clabe: [], fecha: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            clabe: [ '["002180000000000002", "002180000000000002"]' ],
            fecha: [ '[MaxKey, MinKey]' ]
          }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 9,
    totalKeysExamined: 10,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 10,
      executionTimeMillisEstimate: 5,
      works: 11,
      advanced: 10
```

Plan con índice. Se utiliza IXSCAN y se examinan 10 documentos

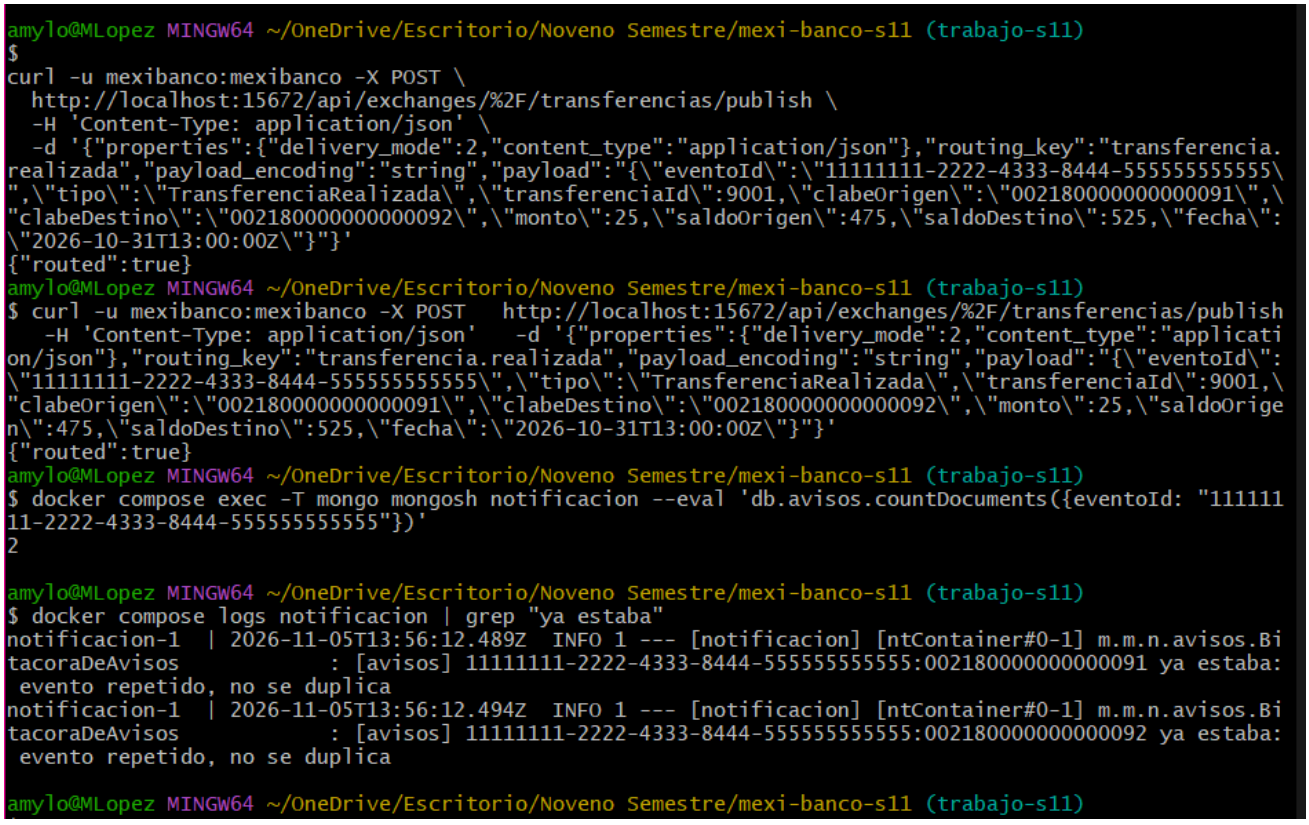
```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"
}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 4,
    totalKeysExamined: 0,
    totalDocsExamined: 28,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 40,
      advanced: 10,
      needTime: 29,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
```

Plan sin índice. Se utiliza COLLSCAN y se examinan 28 documentos

4. Prueba de evento duplicado

Publiqué dos veces el mismo evento de transferencia en RabbitMQ. En ambas ocasiones obtuve routed:true. MongoDB conservó solamente dos avisos, uno por destinatario, y en los logs aparecieron dos mensajes que decían "ya estaba". La combinación eventoId:clabe como _id evitó duplicar el mismo aviso

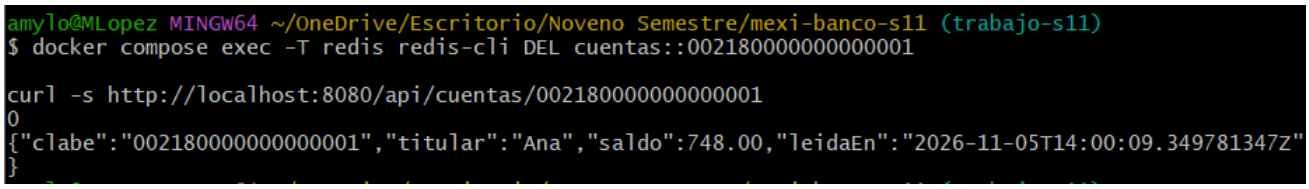
Comprobación	Resultado
Publicaciones del mismo evento	2 respuestas routed:true
Avisos guardados para el evento	2
Avisos duplicados añadidos	0



Publicación duplicada, conteo de 2 y registros de eventos repetidos

5. Prueba de saldo desactualizado

Cargué en Redis una copia del saldo de Ana ($748) y cambié directamente el saldo en PostgreSQL a $5. Mientras duró la copia, el GET mostró el saldo anterior. Después de que expiró su TTL, las consultas devolvieron $5 y un nuevo leidaEn

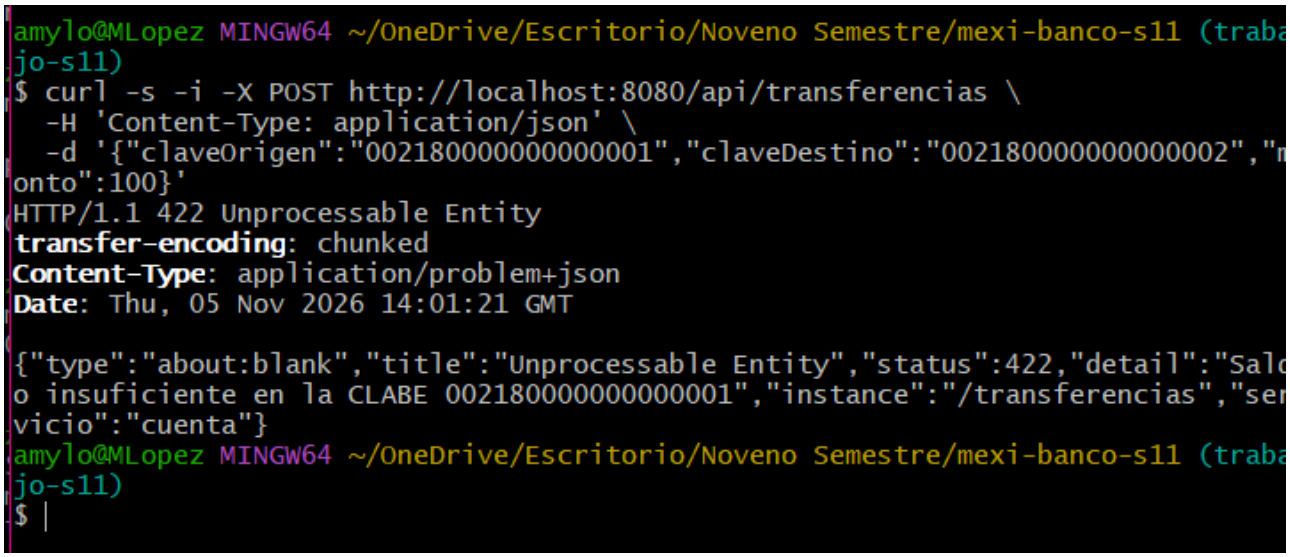


Saldo original de Ana registrado en Redis antes de la modificación directa



El TTL llega a su vencimiento y el saldo mostrado cambia de $748 a $5

Al intentar transferir $100, la API rechazó la operación con HTTP 422 por saldo insuficiente. Repetí la prueba con TTL positivo (25 segundos), Redis seguía mostrando $748, mientras PostgreSQL tenía $5 y la transferencia volvió a rechazarse. Al finalizar restablecí el saldo de Ana a $748 y eliminé la copia anterior



Primera comprobación de rechazo por saldo insuficiente (HTTP 422)

```
amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "UPDATE cuenta SET saldo = 748 WHERE clabe =
 '002180000000000001';"

docker compose exec -T redis redis-cli DEL cuentas::002180000000000001
UPDATE 1
0

amylo@MLopez MINGW64 ~/OneDrive/Escritorio/Noveno Semestre/mexi-banco-s11 (trabajo-s11)
$
# Guardar el saldo de 748 en Redis
curl -s http://localhost:8080/api/cuentas/002180000000000001; echo

# Cambiar PostgreSQL a 5
docker compose exec -T db psql -U mexibanco -d cuenta -c "UPDATE cuenta SET saldo = 5 WHERE clabe = '00
2180000000000001';"

# Verificar que Redis aun muestra 748
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000001; echo

# Intentar transferir 100 inmediatamente
curl -s -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100}'
{"clabe":"002180000000000001","titular":"Ana","saldo":748.00,"leidaEn":"2026-11-05T14:07:02.602479122Z"
}
^NUPDATE 1
25
{"clabe":"002180000000000001","titular":"Ana","saldo":748.00,"leidaEn":"2026-11-05T14:07:02.602479122Z"
}
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Thu, 05 Nov 2026 14:07:12 GMT
```

Prueba decisiva. TTL positivo y saldo viejo de $748 junto con HTTP 422

6. Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Porque los avisos pueden tener campos distintos según el canal, el SMS usa teléfono y texto, correo usa asunto y cuerpo, y push utiliza un token. MongoDB nos permite guardar cada documento con su propia forma. Mientras que, el saldo es dinero real y las operaciones necesitan transacciones confiables, por eso debe permanecer en PostgreSQL

2. ¿Qué hit ratio medí y qué gané con la caché?

En el escenario A medí 99 % y en el B 83.33 %. Bajó porque las transferencias borran los saldos guardados y la siguiente consulta vuelve a PostgreSQL. Con Redis obtuve medianas menores, pero la mayor ventaja fue disminuir las lecturas a PostgreSQL, fueron 3 con caché frente a 300 sin caché en las tres corridas, es por ello que considero que principalmente gané capacidad

3. ¿Qué ocurriría si la autorización leyera el saldo de Redis?

Podría autorizar un cargo con un saldo viejo y esto lo comprobé cuando Redis mostraba $748 aunque PostgreSQL ya tenía $5. Una transferencia de $100 no se completó y devolvió HTTP 422 porque la decisión se tomó con el dato de PostgreSQL

4. ¿Por qué DEL y no SET, y por qué después del commit?

Se elimina la copia con DEL para que la siguiente consulta vuelva a leer el saldo ya confirmado de PostgreSQL, en vez de intentar mantener dos saldos iguales manualmente. También se espera al commit, si la copia se borrara antes, otra petición podría consultar el dato anterior y guardarlo de nuevo en Redis mientras la transacción todavía no termina

7. Conclusión

Aprendí que un sistema distribuido puede usar distintas bases de datos dependiendo de lo que guarda.

PostgreSQL conserva las operaciones con dinero, Redis nos ayuda a responder las consultas de saldo y MongoDB permite guardar los avisos en formatos diferentes.

También comprobé que es necesario evitar notificaciones repetidas y no confiar en una copia del saldo al autorizar una transferencia.